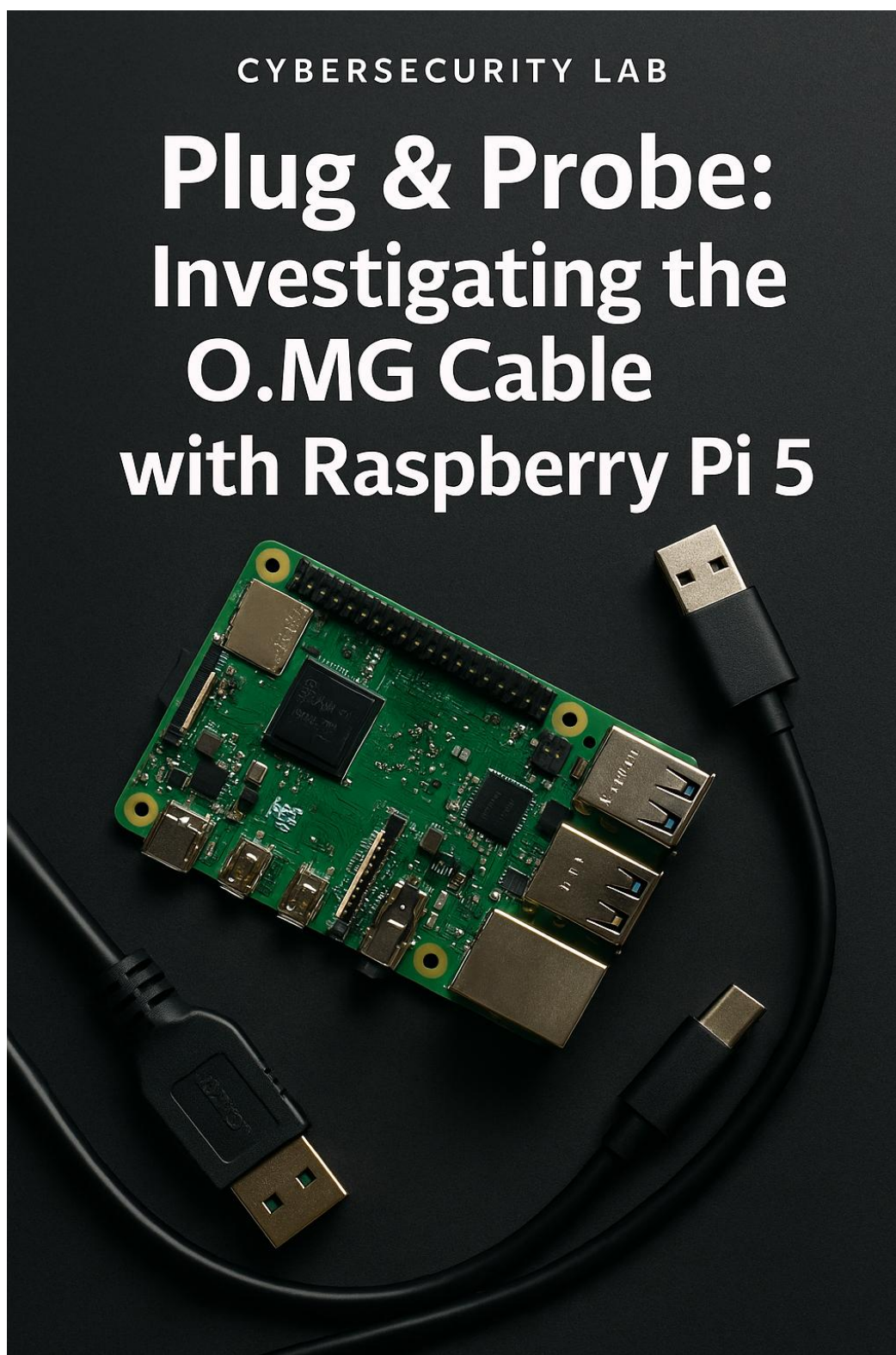


Lab 0 – Plug & Probe: Investigating the O.MG (OMG) Cable with Raspberry Pi 5



Lab 0 – Technical Information Table

Instructor	: Prof. Reda Mastouri	Duration	: ~ 2 hours (setup + testing + discussion)
Course / Program	: Cybersecurity - Associate Degree Program, HCCC	Location	: Cybersecurity Lab @ the 6 th floor- STEM Building, Hudson County Community College (HCCC)
Lab Title	: Plug & Probe: Investigating the O.MG Cable with Raspberry Pi 5	Date & Time	: Tuesday, Sept 23, 2025 - 4:00 PM EST
Lab ID#	: 0 (Introductory hands-on lab)	Lab Requirements	: • Basic Linux terminal skills • Awareness of safe lab practices • Ethical responsibility
Lab Materials	• Raspberry Pi 5 (with SD card, power, HDMI, keyboard/mouse) • O.MG Cable (lab-owned) • Target test laptop/VM with dummy accounts • Wi-Fi/Ethernet (isolated lab network) • Tools: Raspberry Pi OS, <i>lsusb</i>, <i>dmesg</i>, Wireshark/tcpdump, Python 3		

Lab Agenda

1. **Safety & Ethics Briefing** - Define scope: offensive testing *only in lab*, never on personal/production devices.
2. **Introduction to Raspberry Pi 5** - Why it's a flexible red-team testing platform.
3. **Introduction to the O.MG Cable** - Offensive capabilities (keystroke injection, payload delivery, wireless control) and implications for attackers.
4. **Raspberry Pi 5 Setup** - Flash OS, enable monitoring tools, and prepare Pi for offensive payload development.
5. **Hands-On Offensive Testing** -
 - a. **Payload 1:** HID keystroke injection (e.g., launch calculator).
 - b. **Payload 2:** Automated file access / dummy exfiltration.
 - c. **Payload 3:** Wireless control of O.MG cable (if supported).
6. **Detection & Defense (Blue-Team Lens)** - Observe device enumeration, network activity, and log indicators.
7. **Conclusion & Wrap-Up** - Discuss red-team vs. blue-team perspectives and career relevance.

Abstract

This lab is designed as an **offensive security exercise** where students will **explore, deploy, and test** the capabilities of a malicious USB device – the **O.MG cable** – using a **Raspberry Pi 5** as their operational base.

From an **attacker's perspective**, students will learn how the O.MG cable can perform **keystroke injection, launch payloads, and create hidden wireless channels** for command and control. They will use the Raspberry Pi to configure, monitor, and control offensive scenarios in a **safe, isolated environment**.

From a **defender's perspective**, students will practice detecting malicious USB activity through Linux tools (*lsusb, dmesg*) and network monitoring (*Wireshark/tcpdump*).

At the end of the lab, students will:

- Build operational knowledge of Raspberry Pi 5 for offensive testing.
- Execute controlled O.MG cable payloads against a sandboxed target.
- Identify forensic traces left by malicious USB devices.
- Develop dual insights: how attackers think and how defenders respond.

1 Safety & Ethics Briefing

Purpose and Scope

The purpose of this briefing is to define the ethical and operational boundaries for Lab 0. This lab involves controlled offensive security demonstrations using a **Raspberry Pi 5 and an O.MG cable**. All experiments are strictly confined to the college's cybersecurity lab environment. Offensive testing outside of this scope—including personal laptops, mobile devices, or production systems—is not permitted under any circumstances.

Students should recognize that while the techniques introduced in this exercise mirror those used by adversaries, they are to be employed here **only as part of supervised, academic training**. Attempting to reproduce these methods outside the approved environment may violate HCCC's code of conduct, institutional policies, and applicable law.

The Lab Environment

To ensure safety, the lab is segmented into three functional domains that together form the high-level architecture of the experiment (Figure 1).

- The **Red-Team domain** contains the Raspberry Pi 5 (configured as the attacker's staging system) and the O.MG cable, which is the device under test. These are the only components from which offensive payloads originate.
- The **Incident Response & Management (IRM) domain** houses the instructor's monitoring laptop with Wireshark, a central logging/forensics server, the managed lab switch, and a dedicated firewall/router. This environment provides visibility, evidence collection, and network control.
- The **Blue-Team domain** consists of the sandboxed victim system (a virtual machine or test laptop). This is where the O.MG cable is connected and where benign demonstrations of malicious behavior can be safely observed.

All three domains are bound together within VLAN 50, a network isolated from campus and production resources. The router at the edge enforces strict ACLs, permitting no egress except for controlled instructor management traffic.

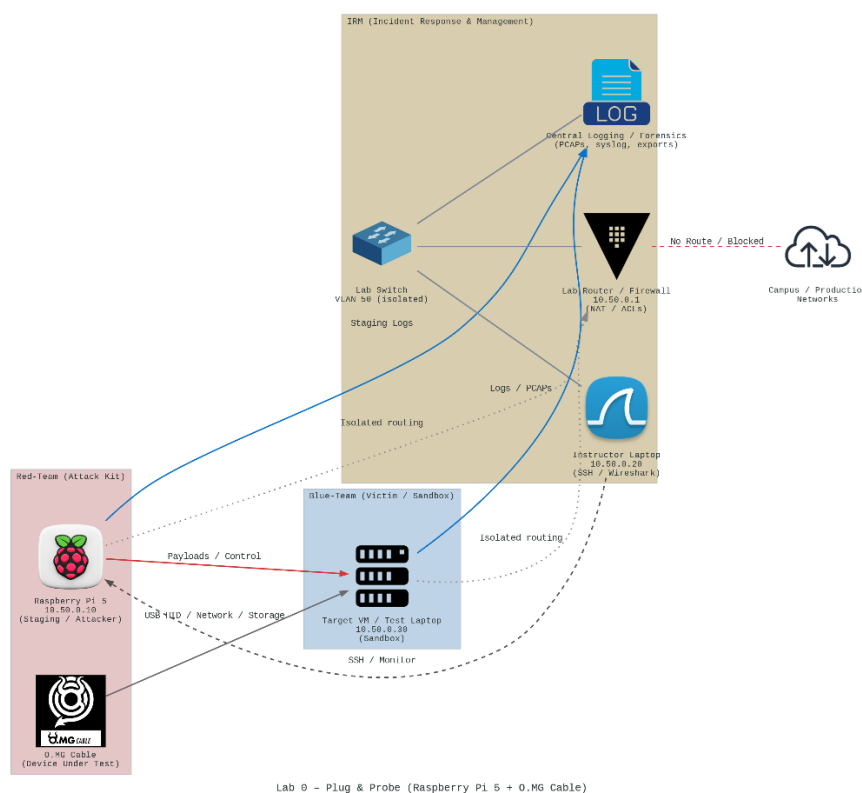


Figure 1 - High-Level Architecture for Lab 0 (Red-Team, IRM, and Blue-Team domains)

Ethical Conduct in Practice

Every student participating in the lab is expected to follow professional standards of ethical conduct. Offensive techniques demonstrated in this environment must not be attempted outside of it. Real credentials, personal accounts, or sensitive institutional data are never to be used. Payloads are prepared and executed only under instructor supervision. Each student's role is to observe, document, and analyze outcomes, not to improvise or extend the attack beyond the agreed scope.

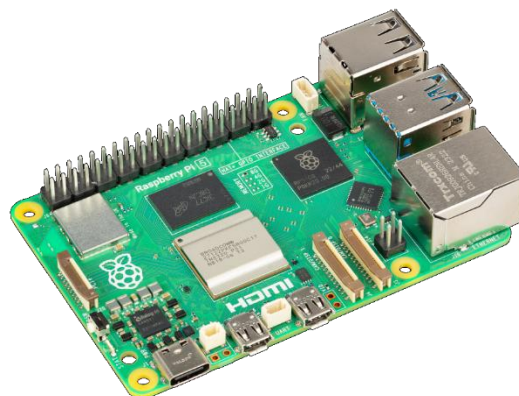
Operational Safety Measures

Before a session begins, the instructor verifies that the environment is correctly staged: VLAN 50 is active and isolated, the target virtual machine snapshot is intact, logging and packet capture tools are operational, and the O.MG cable has been inventoried. Students are briefed on their responsibilities, reminded of the ethical boundaries, and asked to sign an acknowledgment form affirming their understanding.

2 Introduction to Raspberry Pi 5

What is a Raspberry Pi 5?

The Raspberry Pi 5 is a **single-board computer** – meaning it is a fully functional computer condensed onto a circuit board about the size of a deck of cards. Despite its small size, it can run a full operating system, handle networking, and interact with USB devices. In this lab, we use it as the **Red-Team staging host**: the place where payloads are prepared, logs are stored, and monitoring commands are run.



Think of it as a **miniature laptop without a screen or keyboard**. You interact with it remotely, but it has the same parts as a regular computer: processor, memory, storage, and ports. Because of its small size and low cost, attackers often experiment with such devices – and that makes it a perfect teaching tool for us.

Example analogy

If a normal laptop is a full kitchen, the Raspberry Pi 5 is a tiny kitchenette: it can cook simple recipes and show you how cooking works, but it's not a full commercial kitchen. That makes it perfect for learning and demonstrations in a classroom.

Raspberry Pi 5 Technical Specifications

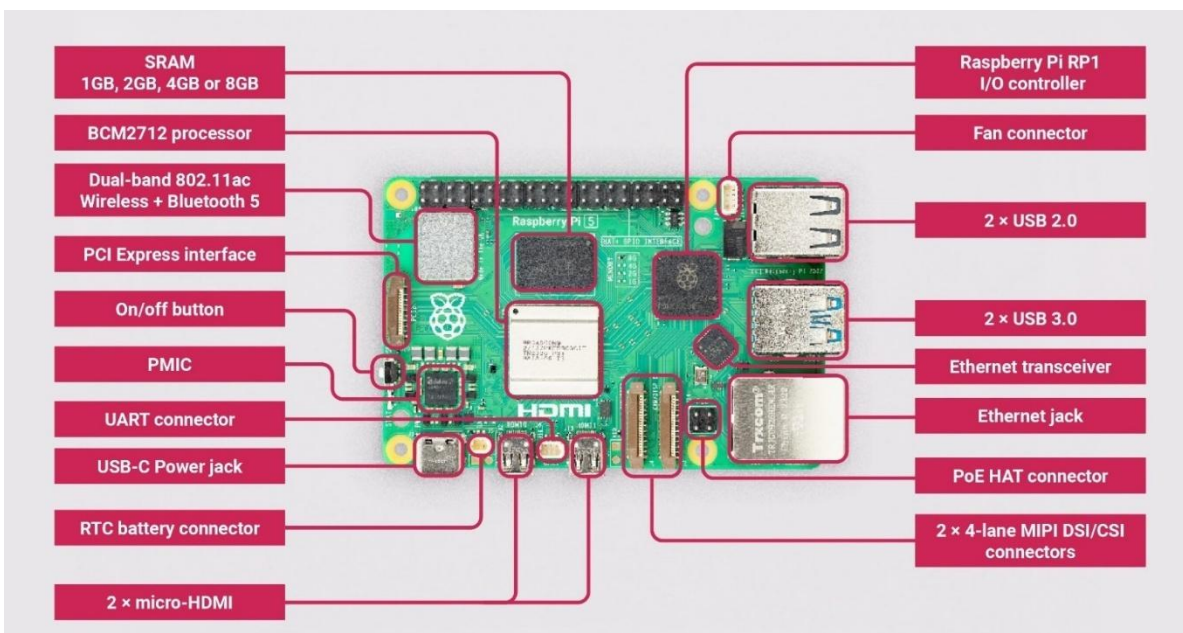


Figure 2 – Source: hackatronic.com

Component	Specification
CPU	Broadcom BCM2712, 2.4 GHz quad-core 64-bit Arm Cortex-A76, cryptography extensions, 512 KB per-core L2 cache, 2 MB shared L3 cache
GPU	VideoCore VII, supports OpenGL ES 3.1 and Vulkan 1.2
Display	Dual HDMI outputs (up to 4K at 60 fps with HDR)
Decoder	4Kp60 HEVC hardware decoder
Memory	LPDDR4X-4267 SDRAM (models with 4 GB or 8 GB)
Wireless	Dual-band 802.11ac Wi-Fi, Bluetooth 5.0, Bluetooth Low Energy (BLE)
Storage	microSD card slot (SDR104 mode)
USB	2 × USB 3.0 ports (5 Gbps each), 2 × USB 2.0 ports
Networking	Gigabit Ethernet, Power over Ethernet Plus (PoE+ with add-on HAT)
Camera/Display	2 × 4-lane MIPI transceivers
Expansion	PCIe 2.0 ×1 interface (via adapter), 40-pin GPIO header
Power	5 V / 5 A USB-C with Power Delivery
Other	Real-Time Clock (RTC, requires external battery), physical power button

Table 1 –Specifications of Raspberry Pi 5



What this means in practice

- The CPU and memory make it powerful enough to act like a small server, staging payloads and running monitoring tools.
- The USB ports let it communicate with devices like the O.MG cable.
- The Gigabit Ethernet keeps it fast and reliable when connected to the isolated lab VLAN.
- The Wi-Fi and Bluetooth are normally disabled in this lab – we don't want signals leaking outside.
- The 40-pin GPIO header and PCIe slot allow advanced projects, but we won't use them in this course.



Role in the Lab Architecture

In our high-level setup (see Figure 1: Lab Architecture and Figure 2: Pi5 Staging Flow), the Raspberry Pi 5 sits in the **Red-Team domain**. It is assigned IP address **10.50.0.10** and connects directly to the O.MG cable. Logs from the Pi are pushed to the **IRM log server** so students can analyze the attack evidence without controlling the attack themselves.

3 Introduction to O.MG Cable

🔌 What is an O.MG cable?



The O.MG cable is a cleverly disguised offensive tool. To an unsuspecting user, it looks like a normal USB charging or data cable – something you might casually plug into a laptop or phone without a second thought.

Inside its connector housing, however, are carefully hidden electronics: a microcontroller, a storage chip, and sometimes even a small antenna. These parts allow the cable to act like a programmable attack platform. In the same way that the Raspberry Pi 5 functions as a miniature computer on a circuit board, the O.MG cable is a **computer hidden in a cable**. In our lab, it is used strictly for controlled demonstrations so that students can see how

malicious hardware behaves and how defenders can detect it.

🔍 Example analogy

At first glance, the **O.MG Cable** looks no different from the charging cords you carry every day. But beneath its plastic jacket lies a **microprocessor, antenna, and hidden chip** (see Figure 3). These hidden components transform the cable from a simple data/charging conduit into an **offensive cyber tool** capable of executing commands, injecting keystrokes, or even creating covert wireless links. Think of it as the **"Trojan horse"** of USB accessories: harmless in appearance, but secretly weaponized.

📄 Anatomy of the O.MG Cable

Industrial CT scans and reverse-engineering efforts reveal just how much sophistication is crammed inside this tiny device:

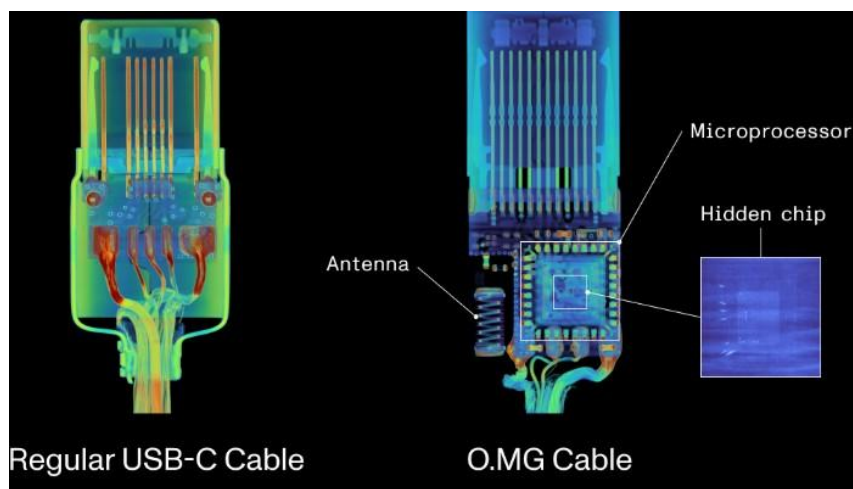


Figure 3 – CT scan by Lumafield. Investigating Malicious Hardware with Industrial CT. December 2024.

The table below summarizes the internal components of the O.MG cable and what each one contributes to its behavior:

Component	Role inside the cable
Microprocessor	The “brain” that runs firmware, interprets triggers, and executes pre-loaded instructions.
Hidden storage chip	Small nonvolatile memory that stores scripts, payloads, and configuration.
Antenna (wireless models)	Enables remote control and wireless command-and-control (C2) from a short range.
HID firmware capabilities	Firmware routines that allow the cable to present as an HID (keyboard/mouse) or other USB class.
Normal USB wiring & passthrough	The data/charging conductors that let the cable behave like a regular cable while masking the hidden electronics.

Table 2 – Anatomy of the O.MG Cable

💡 Why These Elements Matter for Our Lab

Each internal component has a direct effect on what defenders can observe. The **microcontroller and storage** let the cable behave autonomously: once plugged in, it can run preloaded instructions without needing another device nearby.

The O.MG Cable supports multiple attack modes that red-teamers and penetration testers can configure:

1. **HID Keystroke Injection** – Cable pretends to be a keyboard, typing commands at lightning speed into the victim machine.
2. **Wireless Command & Control (C2)** – Attacker can connect via Wi-Fi and trigger payloads remotely.
3. **Keylogging** – Cable silently records keystrokes and stores or exfiltrates them.
4. **Scripted Payload Delivery** – Preloaded sequences can automate exploitation (e.g., opening a shell, downloading tools).
5. **Stealth Operation** – Because the cable still works as a normal charging/data cable, most users won’t suspect it.

🖼️ The hidden design revealed

Figure 3 shows a CT scan of an O.MG cable connector compared with a normal USB connector. The scan reveals just how precisely the malicious components are arranged: the storage chip is bonded to the microcontroller with wires thinner than a human hair, and the antenna is integrated so seamlessly that ordinary inspection tools would miss it. This illustrates a central lesson: **malicious hardware can look completely benign from the outside**, which is why defenders must rely on logs, monitoring tools, and security policies rather than appearances.

 How the O.MG cable is used in this lab

In our **Lab 0 environment**, the O.MG Cable is placed under controlled conditions:

- **Red-Team** uses the cable to simulate real-world offensive techniques.
- **Blue-Team** monitors how the victim machine behaves when targeted.
- **IRM** captures forensic evidence (logs, PCAPs) to practice detection.

This cable is dangerous in the real world, but here it is **isolated within VLAN 50** (see Figure 2 – High-Level Architecture) and handled strictly as a **training tool**.

 Safety Reminder

The O.MG cable used in this course is lab equipment. It must **never be plugged into personal devices** such as laptops or phones. Doing so could cause data loss or policy violations. Its purpose here is strictly to **teach detection and defense in a controlled environment**.

4 Raspberry Pi 5 Setup

Overview & purpose

The Raspberry Pi 5 in Lab 0 functions as the **attacker staging node**: it hosts tooling and staging artifacts, controls or configures the O.MG cable hardware, runs lightweight services (for staging or receiving test data), and forwards logs and telemetry to the IRM (central logging) plane. In our architecture the Pi is intentionally **isolated on VLAN 50** (IP 10.50.0.10) and accessible only by the instructor (10.50.0.20) via SSH. We treat the Pi as a *sensitive, auditable device* – every action on it is logged, monitored, and reversible.

Ready-to-use downloads & tools

Component	Description	Resources
Raspberry Pi OS (official)	Download the supported Pi OS images and read the official imaging guidance from the Raspberry Pi Foundation. Use the 64-bit image for Pi 5.	 Click here
Raspberry Pi Imager	The recommended, official imaging utility for writing Pi OS to microSD cards (Windows/macOS/Linux). Use this for reproducible, consistent lab images.	 Click here
BalenaEtcher	An alternative cross-platform imaging tool (useful if Imager isn't available).	 Click here
O.MG Cable (<i>product page / vendor</i>)	Product description and defensive/educational use context (Hak5 shop). Note: this is a lab tool for detection exercises only; treat as sensitive hardware.	 Click here
Incident response best practice	NIST SP 800-61 (incident handling guide) for lab IR workflow and evidence handling. Use it as the formal reference for your IRM procedures.	 Click here

Table 3 – Lab 0 Resources

Pi 5 role diagram

Pi responsibilities:

- host staging artifacts (read-only for students),
- capture packet traces (**tcpdump**) and keep a time-stamped log bundle,
- forward logs to central logging server (secure **SCP/rsync**),
- provide a controlled service endpoint for safe demonstration (instructor-only),
- be fully auditable (**systemd journal** + persistent log directory).

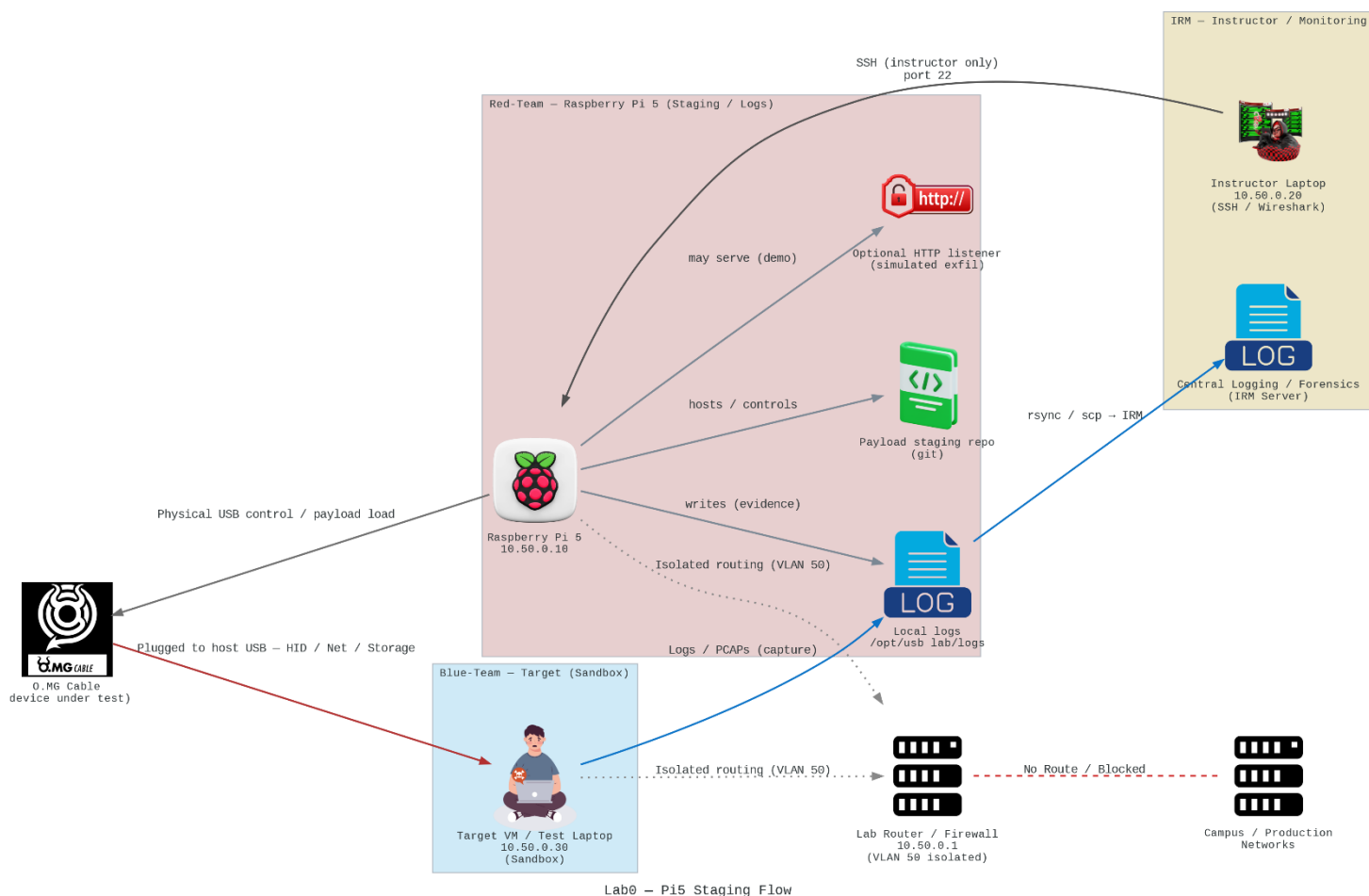


Figure 4 – Pi5 Staging Flow

🔧 Imaging and first-boot (concise, reproducible)

Use the official imager to create a **golden microSD card**. The steps below are what you will perform once – the result becomes your *golden image* you'll clone for all lab Pi units.

1. On a workstation, download the Pi OS image from Raspberry Pi Foundation and run **Raspberry Pi Imager** (or **balenaEtcher**) to flash the image to a microSD card. (See Raspberry Pi OS and Imager links in **table 3.**)
2. To enable headless SSH for initial configuration, follow the Raspberry Pi documentation: either enable SSH in Imager or place an empty file named `ssh` in the boot partition before first boot.
3. On first boot log in as the initial user, create the instructor account, lock or remove the default pi user, and set a strong SSH key for the instructor (see SSH hardening below).

Imaging & first-boot (Instructor steps)

on local console or via SSH, change default password and create an instructor admin user:

```
# after first login as 'pi'
sudo passwd pi          # set a strong password if you continue using 'pi'
# create an instructor user
sudo adduser instructor
sudo usermod -aG sudo instructor
# disable plain 'pi' use (recommended)
sudo passwd -l pi
```

Let's now set the timezone & locale:

```
sudo timedatectl set-timezone America/New_York
sudo dpkg-reconfigure locales    # set en_US.UTF-8
```

Minimal hardening & network placement (instructor-only)

The Pi sits on VLAN 50 (isolated, 10.50.0.0/24) with a static address assigned 10.50.0.10. The router enforces *no route to production*; only instructor IP (10.50.0.20) is permitted SSH.

Key config steps (concise commands – instructor executes):

- Set static IP (example using **dhcpcd.conf**):

```
# append to /etc/dhcpcd.conf
interface eth0
static ip_address=10.50.0.10/24
static routers=10.50.0.1
static domain_name_servers=10.50.0.1
sudo systemctl restart dhcpcd
```

- Allow only instructor key and disable password auth by generating SSH key (if not existing):

```
ssh-keygen -t ed25519 -C "instructor@hccc"
ssh-copy-id -i ~/.ssh/id_ed25519.pub instructor@10.50.0.10
```

- SSH key-only access and user hardening:

```
# on instructor laptop:
ssh-keygen -t ed25519 -f ~/.ssh/id_usblab
ssh-copy-id -i ~/.ssh/id_usblab.pub instructor@10.50.0.10
```



```
# on Pi, /etc/ssh/sshd_config (important lines)
```

```
PermitRootLogin no
PasswordAuthentication no
ChallengeResponseAuthentication no
AllowUsers instructor
PubkeyAuthentication yes
PermitEmptyPasswords no
UsePAM yes
```

```
# restart ssh
sudo systemctl restart ssh
```

- Basic host firewall (UFW) restricting incoming traffic to instructor IP – restrict services to lab VLAN only:

```
sudo apt update && sudo apt install -y ufw
sudo ufw default deny incoming
sudo ufw default allow outgoing

# allow SSH only from instructor IP
sudo ufw allow from 10.50.0.20 to any port 22 proto tcp

# allow NTP or DNS to proxy if needed
sudo ufw enable
```

These controls preserve the Pi's role as a *controlled* staging host while allowing the instructor to manage demonstrations remotely.



Required packages and safe toolset

Install only the monitoring, logging, and benign tooling needed for capture and analysis:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git python3 python3-pip tcpdump tshark usbutils htop
rsync
sudo pip3 install pyusb requests
```

- usbutils / lsusb provides USB device enumeration and vendor/product lookups (essential for detecting device classes and vendor IDs).
- tcpdump / tshark capture network activity for later analysis. Use short, instructor-triggered captures to avoid storage blowup.

📁 Logging & evidence handling (best practice)

Keep a standardized, timestamped folder for every demonstration; make copies to an immutable, central forensics store.

Create a standardized log directory with strict permissions. Instructor should periodically archive logs.

```
sudo mkdir -p /opt/usb_lab/logs
sudo chown instructor:instructor /opt/usb_lab/logs
sudo chmod 750 /opt/usb_lab/logs
```

Recommended layout:

```
/opt/usb_lab/logs/2025-09-23T1600Z/
lsusb_before.txt
dmesg_before.txt
lsusb_after.txt
dmesg_after.txt
capture.pcap
notes.txt
```

Use a small script to collect artifacts before/after each run (example `usb_logsnap.sh` described in your runbook). Always move the run folder to the IRM server and mark it immutable there.

For aggregation: the IRM server ingests PCAPs and logs so students can analyze indicators centrally – align this practice with incident-handling procedures found in NIST SP 800-61.

Automatic rotation & retention using `logrotate` or a cron job – keep logs for e.g., 90 days then archive.

Example `logrotate` snippet `/etc/logrotate.d/usb_lab`:

```
/opt/usb_lab/logs/*/*.pcap {
    rotate 6
    weekly
    compress
    missingok
    notifempty
}
```

📄 Snapshot & evidence snapshot script (recommended)

Create a simple reproducible snapshot script that collects the critical artifacts at run time.

/usr/local/bin/usb_logsnap.sh:

```
#!/bin/bash
TS=$(date -u +"%Y%m%dT%H%M%S")
DIR="/opt/usb_lab/logs/$TS"
mkdir -p "$DIR"
lsusb -v > "$DIR/lsusb_full.txt" 2>&1
lsusb > "$DIR/lsusb.txt"
dmesg | tail -n 500 > "$DIR/dmesg_tail.txt"
ip a > "$DIR/ip_a.txt"
uname -a > "$DIR/uname.txt"
# optional: short tcpdump (instructed-run only)
sudo timeout 30 tcpdump -i any -s 0 -w "$DIR/capture.pcap"
echo "Saved logs to $DIR"
```

Make it executable:

```
sudo chmod +x /usr/local/bin/usb_logsnap.sh
```

Monitoring & inspection commands (quick, do not alter payloads)

These commands are for observing device presentation and kernel events. Run them before and after connecting the cable:

- USB enumeration:

```
lsusb -v > ./lsusb_full.txt
lsusb > ./lsusb.txt
```

- Kernel messages:

```
dmesg | tail -n 200 > ./dmesg_after.txt
```

- Packet capture (instructor-triggered, short duration):

```
sudo timeout 30 tcpdump -i any -s 0 -w ./capture_after.pcap
```

The lsusb utility documentation explains vendor/product IDs and how to interpret descriptors; include that doc as required reading for analysis.

⚙️ Safe demo services (instructor-controlled only)

If you require a benign network endpoint (for simulated exfiltration), host a short-lived, local service on the Pi and capture traffic. Example (safe, HTTP for demonstration):

```
# from a controlled instructor directory
cd /opt/usb_lab/exfil
python3 -m http.server 8080 &
sudo timeout 30 tcpdump -i any port 8080 -w
/opt/usb_lab/logs/latest_exfil.pcap
```

Never expose such services beyond VLAN 50. Never place real credentials or sensitive data into the demo service.

✅ Instructor checklist (one-line, to paste in runbook)

Before students arrive: flash golden image, configure static IP 10.50.0.10, provision instructor SSH key, enable UFW rules, verify snapshot/rollback on target VM, test usb_logsnap.sh, and confirm central log server reachable.

✅ Verification & troubleshooting

Verify static IP & VLAN

```
ip a show eth0
ip route
# ping instructor
ping -c 3 10.50.0.20
```

Confirm SSH hardening

```
sudo ss -tuna | grep :22
sudo ufw status
# attempt password login (should fail)
ssh pi@10.50.0.10 # should not accept password
```

USB device enumeration

- Baseline:

```
lsusb > ~/baseline_lsusb.txt
dmesg | tail -n 100 > ~/baseline_dmesg.txt
```

- After device plug:

```
lsusb > ~/after_lsusb.txt  
dmesg | tail -n 200 > ~/after_dmesg.txt
```

Check captures

```
ls -lh /opt/usb_lab/logs/  
sudo tcpdump -r /opt/usb_lab/logs/2025.../capture.pcap -n -tttt | head
```

5 Hands-On Offensive Testing

🔧 Should the Victim be Pi 5 or a Windows/Linux Host?

Victim Choice	Pros	Cons	Limitations
Raspberry Pi 5 as Victim	- 🌱 Self-contained lab: Students only need Pi 5 + O.MG cable (no extra VM setup). - 🐛 Low-level visibility: Can capture raw USB activity directly (dmesg, usbmon, lsusb). - 🛠️ Lightweight: Easy to reset/reimage; low risk of contaminating personal machines. - 🔍 Transparency: Students see exactly how a host enumerates a suspicious USB device.	- 💻 Not representative of corporate endpoints: Enterprises don't usually use Pis as desktops. - ⚠️ Some payloads won't translate (e.g., HID keystrokes like opening Calculator are OS-specific). - 📖 Steeper learning curve: Students must be comfortable with Linux command-line early on.	- Driver/driverless behavior: some USB classes behave differently on ARM/Linux vs x86/Windows (e.g., vendor drivers). - Limited native Windows log artifacts (Event Viewer, Sysmon) for blue-team practice. - Peripherals & HATs variability: different Pi revisions / HATs can affect USB bus layout and monitoring.
Windows/Linux VM as Victim	- 💻 Realistic endpoint: Matches what attackers actually target (user laptops, servers). - 🛡️ Payloads demonstrate clearly (launching apps, credential theft, data exfil). - 🏢 Blue-team relevance: Lets students practice detecting logs/events that a SOC analyst would realistically see. - 🔄 Easy reset: Snapshots let you revert quickly after tests.	- 🖥️ Heavier setup: Requires virtualization or spare PC. - 🚫 Less physical: Students don't see raw USB enumeration, only OS-level effects. - 🏗️ More infrastructure: IRM server and VM networking need careful isolation.	- Virtualized USB passthrough quirks: USB device passthrough to VMs can change VID/PID behavior or hide some low-level signals. - Licensing & software install requirements (Windows images, Sysmon). - Resource & scheduling limits: fewer concurrent VMs per lab compared to many Pis.

Table 4 – Pi 5 VS a Windows/Linux Host as Victim Machine

5.1 Victim Host Scenarios: Where Should the O.MG Cable Plug In?

In any O.MG exercise, a key decision is: *what machine will act as the "victim host"?* The cable must be physically plugged into some USB host for its offensive payloads to unfold. Depending on whether we want to emphasize hardware trust, enterprise realism, or a full end-to-end perspective, there are three teaching paths. Each path is illustrated with a dedicated figure.

◆ 5.1.1 Raspberry Pi 5 as Victim – Fundamentals and Hardware Trust

When the **Pi 5 itself acts as the victim**, students can see first-hand how a seemingly normal cable enumerates on the USB bus. The Pi will report new devices in `dmesg`, `lsusb`, and `usbmon`. This environment is compact, inexpensive, and excellent for building intuition about “how a charger becomes a keyboard, a network card, or a storage stick.”

In this setup, the instructor’s laptop monitors logs and provides oversight, while the Pi serves as the primary victim system. This makes it a perfect entry point for students who are just beginning to explore malicious hardware.

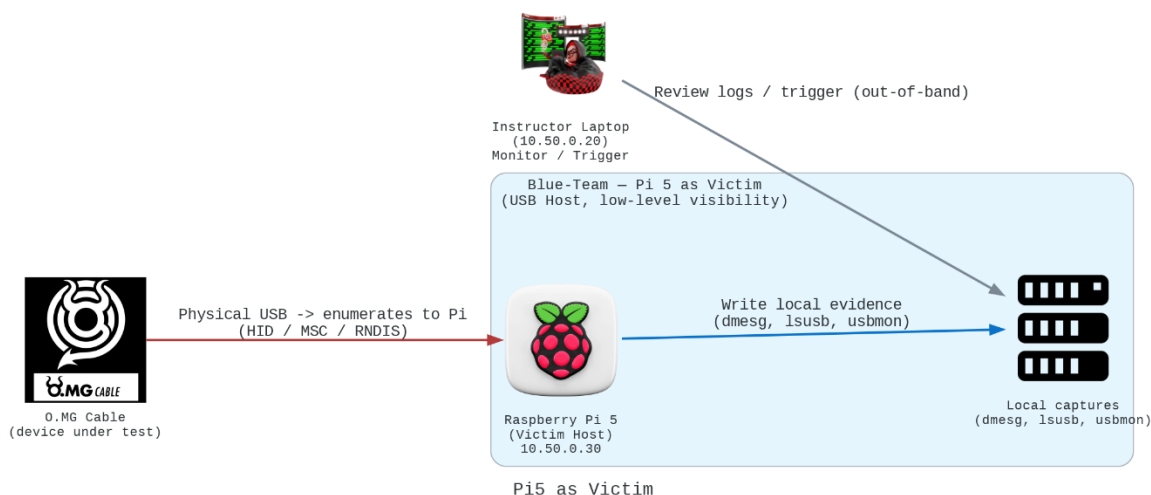


Figure 6a – O.MG cable plugged directly into a Pi 5 victim (hardware trust scenario)

◆ 5.1.2 Windows or Linux VM as Victim – Enterprise Red-Team and Blue-Team Realism

For exercises that must reflect **real corporate environments**, the victim is best modeled as a Windows or Linux VM. This allows students to observe payloads such as application launches, file exfiltration, or persistence in ways that map directly to enterprise SOC workflows. Logs can be collected via Windows Event Viewer, Sysmon, or `journald/systemd` on Linux.

The instructor retains control of the O.MG triggers, but the student’s focus shifts to the defender’s view: what shows up in logs, processes, and alerts when such a device is active in a typical corporate host?

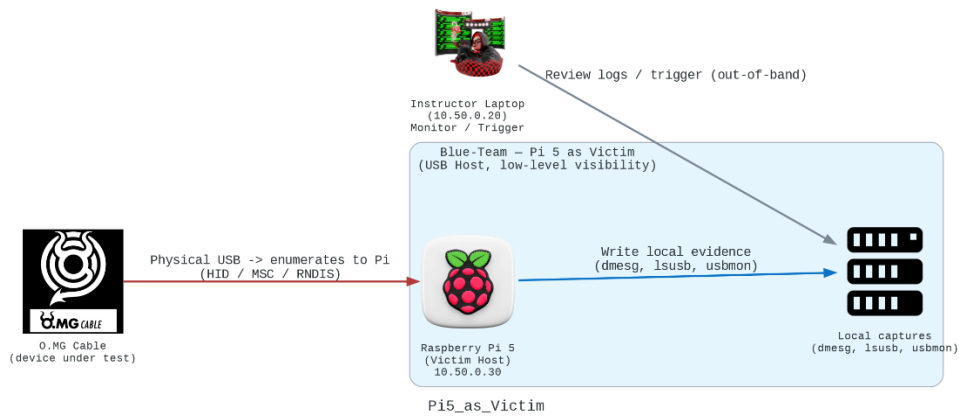


Figure 6b – O.MG cable interacting with a VM victim to simulate enterprise endpoint compromise

5.1.3 Hybrid Approach – Fundamentals First, Then Realism

The **recommended approach** combines both worlds. Students begin with the Pi 5 victim, gaining insight into the raw USB events and enumeration process. Once comfortable, they transition to a VM environment where they can connect what they saw at the hardware layer to enterprise-grade telemetry and blue-team defenses.

This phased sequence gives students the “aha” moment: the same malicious cable that looked suspicious in dmesg on the Pi will also light up Windows Event Viewer or Sysmon in a more realistic target. It also reinforces the idea that hardware trust and enterprise detection are two sides of the same security coin.

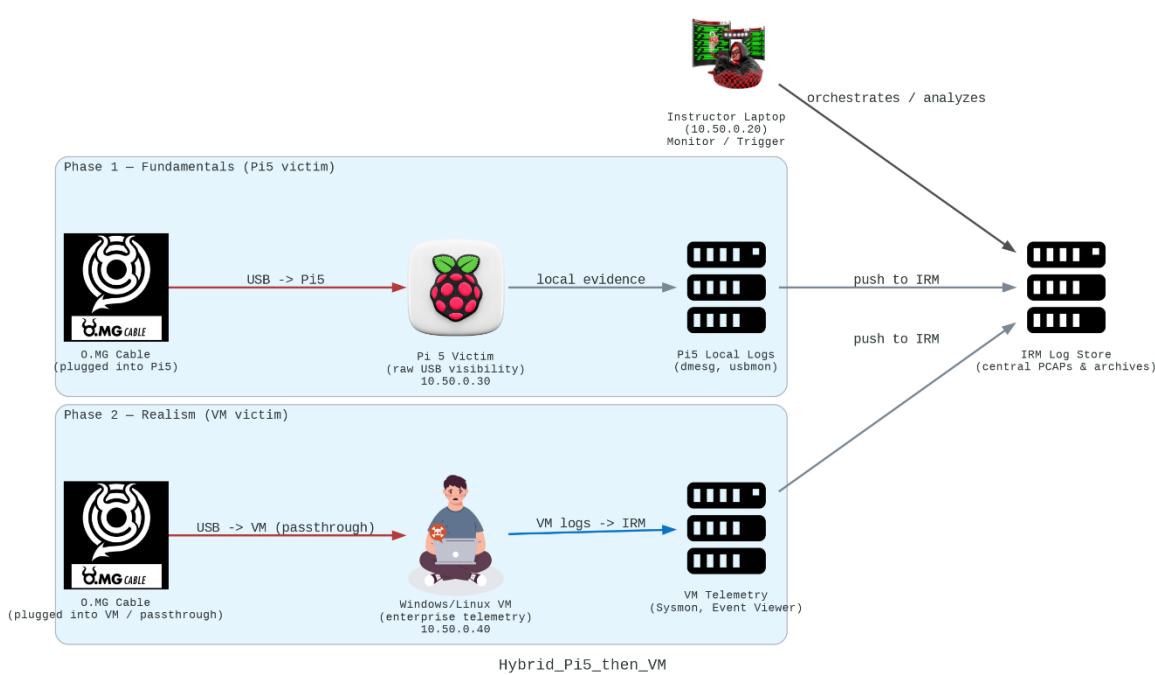


Figure 6c – Hybrid scenario: Phase 1 on Pi 5, Phase 2 on VM, both feeding into IRM logs

The O.MG cable only becomes valuable if students understand its physical role in the testbed. In our lab:

- **Raspberry Pi 5 (10.50.0.10)**: staging host. Holds logs, optional dummy HTTP server for exfil, and can act as a controller. Students SSH into the Pi for observation and artifact collection.
- **Victim VM (10.50.0.30)**: sandboxed Windows or Linux target where the cable is physically plugged in. The “attack” is visible here.
- **Instructor Laptop (10.50.0.20)**: not part of the attack path, but used to monitor IRM logs, trigger the O.MG payload, and verify isolation.

The physical cable connection is always:

Pi5 → O.MG cable → Victim VM

(so the Pi5 can interact with the cable’s management interface while the victim VM treats it like a normal USB cable).

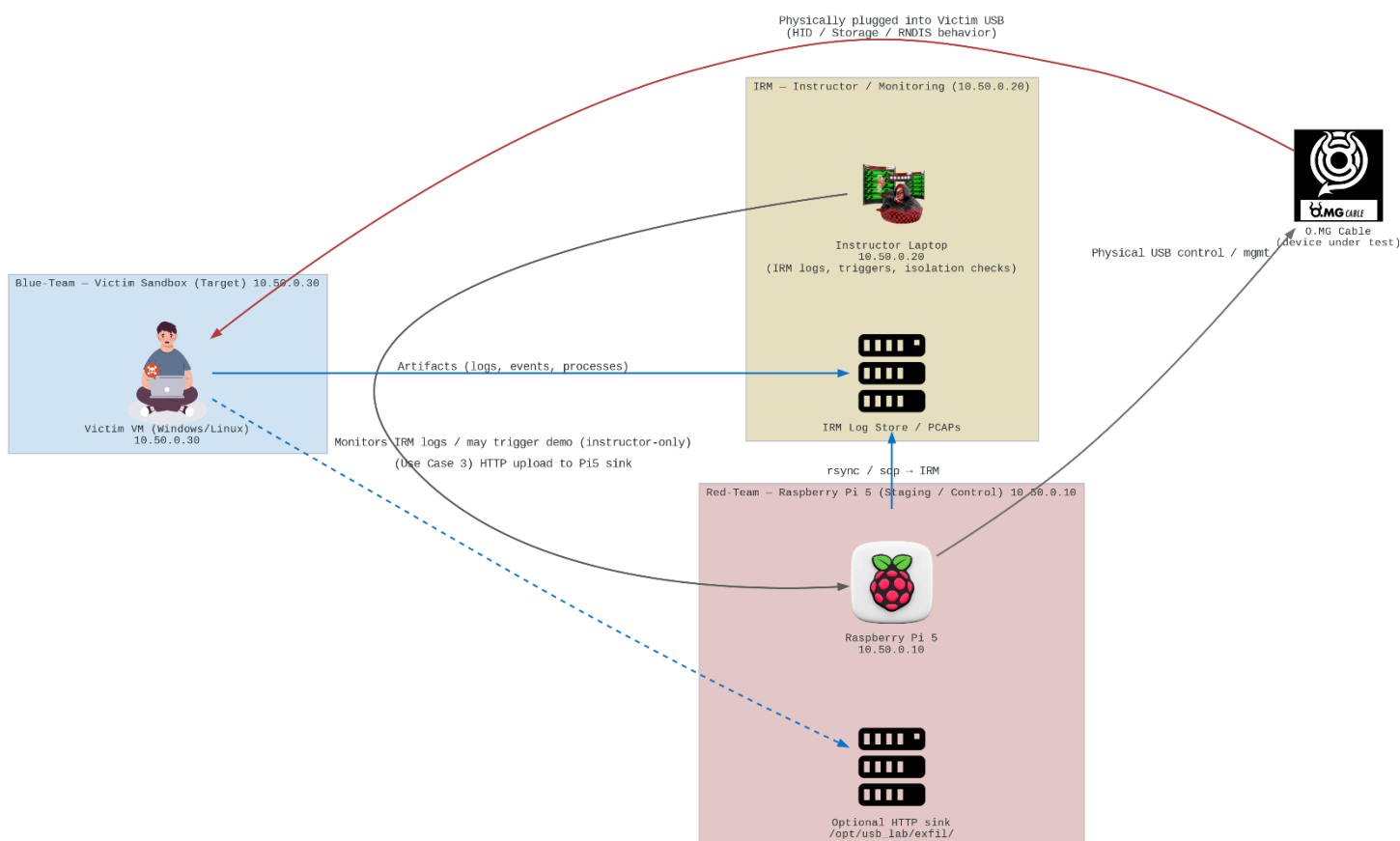


Figure 5 – O.MG Cable Workflow (Pi5 → O.MG → Victim; Instructor monitors)

O.MG management tools

Tool / UI	Purpose (short)	When to use (instructor)	Link
O.MG Web Flasher	Browser-based flasher to connect to the O.MG Programmer and perform an easy firmware flash (3-step Plug → Connect → Flash).	Use for single-device, GUI-driven flashing on the bench when you want a fast, visual workflow. Good for first-time set up.	https://o-mg.github.io/WebFlasher/
O.MG Web UI (device)	The device's runtime web interface (access via http://192.168.4.1 in AP mode or device IP in Station mode). Configure network, check About, enable/disable features, and manage device state.	Use to change SSID/password, inspect firmware version, verify device identity, and trigger benign built-in actions.	(Access after flash via the device SSID / IP shown on success screen)
Hak5 / omg-payloads repo	Community collection of payload examples (HID scripts, benign test payloads, Q-logger readers). Instructor: vet and curate before using.	Use for selecting safe, benign demo payloads (marker writes, LED blink, simple harmless keystrokes). Do not load unreviewed community payloads in class.	https://github.com/hak5/omg-payloads
O.MG Firmware repo & wiki	Official firmware sources, advanced flasher docs, and detailed device docs (including the Python flasher).	Use when you need offline flashing, bulk operations, firmware verification, or advanced configuration. Instructor-only.	https://github.com/O-MG/O.MG-Firmware/wiki/Advanced-Flasher
O.MG Programmer (hardware)	Physical programmer that the Web Flasher talks to (connects to PC via USB and to the	Always use the official programmer for flashing; keep	(Official Hak5 store / product page – include purchase link in procurement section)

	O.MG device). Required for flashing.	it in secure storage.	
Local management host (Raspberry Pi 5 staging)	Pi runs a benign "staging" web service / HTTP sink, stores logs, and acts as the artifact collection point (10.50.0.10 in the lab architecture).	Use for hosting a controlled internal payload sink (dummy HTTP endpoint), storing PCAPs, and point of SSH access for students to collect logs.	(Your internal Pi setup; see lab instructions for /opt/usb_lab/)

Table 6 – O.MG Management Tools

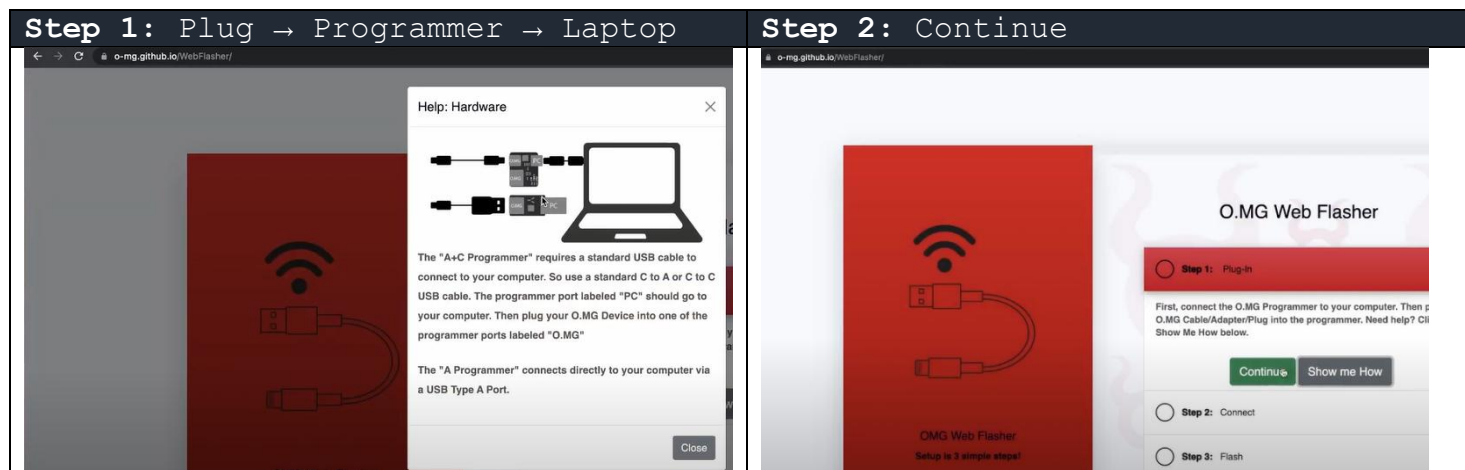
5.2.1 Preflight / checks (do these before touching the device)

- Record procurement info and asset tag for each O.MG device; add an entry in the lab equipment ledger.
- Use an isolated instructor bench on VLAN 50 or physically disconnected Wi-Fi. Disable laptop internet if you will flash with the Web Flasher in AP-only mode.
- Prepare an instructor laptop with Chrome/Edge (Web Serial support) and the Web Flasher page cached/offline if needed.
- Prepare the Pi5 staging host (10.50.0.10) with /opt/usb_lab/logs/ and usb_watch.sh available. Ensure IRM server (10.50.0.20) is reachable from Pi5 for log forwarding.
- Choose and vet the demo payloads you will use – only benign, instructor-approved payloads (e.g., marker file creation, LED blink, simple typed phrase, or a curl to an internal HTTP sink). Document checksums of payload files you intend to load.

5.2.2 Wiring the programmer + cable (what plugs into what)

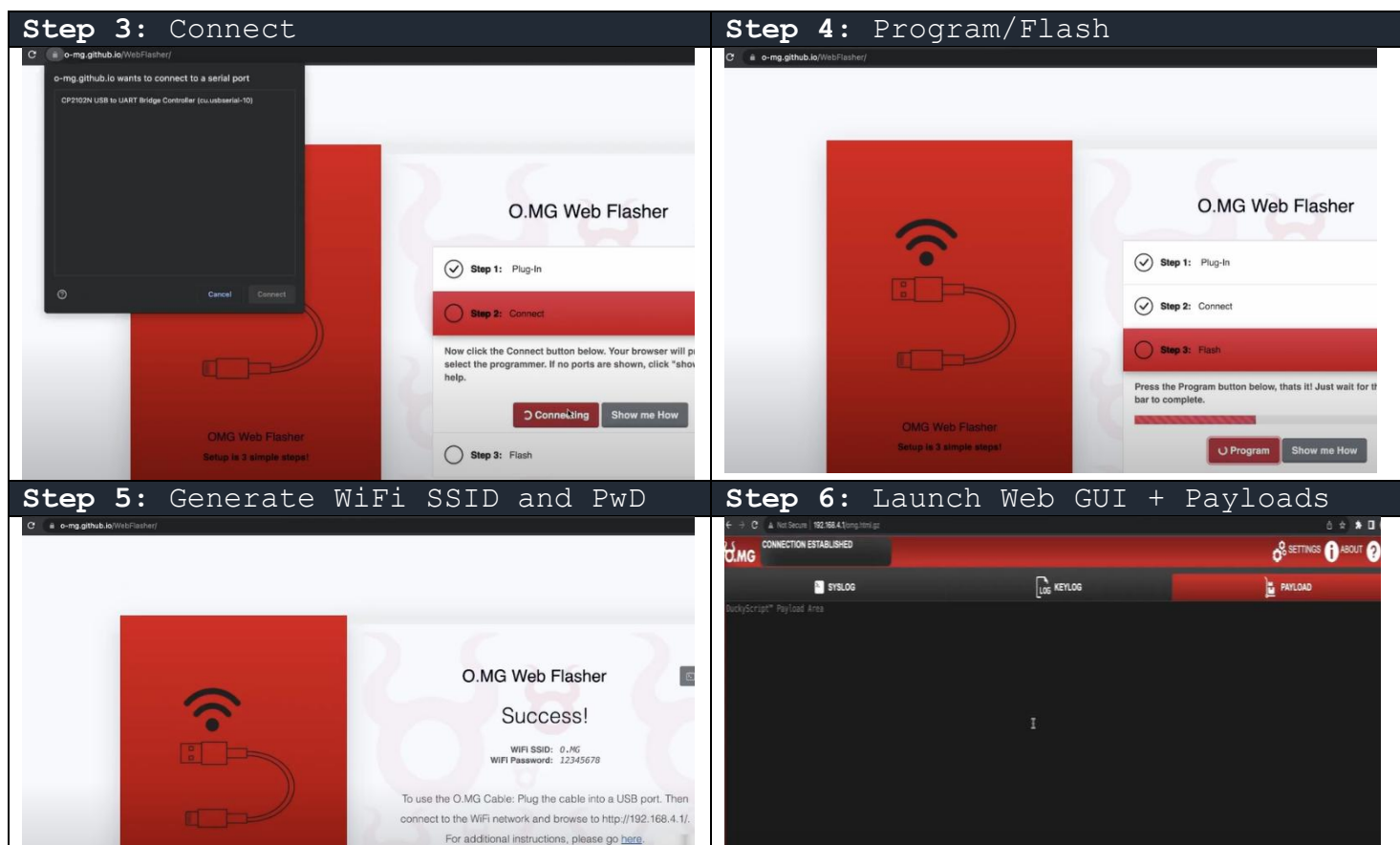
- Connect the **programmer** to the instructor laptop with a data-capable USB cable (**C→C or C→A depending on ports**). The programmer usually has a PC port for the host and one or more O.MG ports for the device.
- Plug the **O.MG device** into the programmer port labeled O.MG (or follow the programmer's help graphic). If using an "**A+C Programmer**" variant, ensure you understand which port is the host/programmer vs device endpoints.
- Verify the programmer is enumerated by the host OS (**serial device like CP210x or cu.usbserial-... appears**). The Web Flasher will list that port.

Diagram note for manual: include a small figure (**Plug → Programmer → Laptop**) and label the ports (PC vs O.MG).



5.2.3 Gaining access to the Web UI (flash → connect workflow)

1. Open the **O.MG Web Flasher** (<https://o-mg.github.io/WebFlasher/>) in Chrome/Edge.
2. Follow the Web Flasher steps: **Plug-In** → **Connect** → **Flash**. Click **Connect**; choose the programmer serial port when the browser prompts.
3. Click **Program** and wait for success. Do not interrupt the process.
4. After success the device typically reboots and advertises an AP (default SSID often O.MG, default password shown on success screen). Record the SSID/password and firmware version in the asset ledger.
5. Connect a **secured instructor laptop** to the O.MG AP (or to the device IP if set to Station mode). Browse to `http://192.168.4.1` (or the IP shown) to reach the device Web UI. Verify **About** (model, firmware, payload version).
6. In the UI, **do not** enable unknown community payload downloads. Set payload source to local or to your vetted repo if possible.



Instructor tip: for classroom demos, either (A) put the device into AP mode and connect the instructor laptop directly to the device Wi-Fi, or (B) set the device to Station mode on a closed lab SSID (VLAN 50) so multiple instructors can reach it via the lab network. Always avoid connecting the device to campus production Wi-Fi.

5.2.4 Adding payloads for the demos (safe, instructor-controlled)

Goal: load only pre-approved, benign payloads that demonstrate HID injection, persistence marker, and HTTP upload to Pi5. Do **not** use the community payloads without vetting.

Instructor workflow (high level):

1. On instructor laptop or Pi5: clone or copy your vetted payload folder (examples: marker_payload/, blink_payload/, exfil_demo/) and compute checksums (sha256). Store checksum values in the asset record.
2. In the **device Web UI**, use the upload function (if available) or use the **Python Advanced Flasher** (offline) to write payload files into device storage. The Web UI may accept payload selections or a payload URL – use only local files or your internal repo. See Advanced Flasher docs if Web UI upload is not used: <https://github.com/O-MG/O.MG-Firmware/wiki/Advanced-Flasher>.

3. After payloads are installed, reboot the device and **verify** via the About / Payload list that the expected payload names and version strings appear. Take screenshots.
4. Secure device: change AP SSID/password to a lab-only value or disable AP mode if you'll use a station on VLAN 50. Document settings.

Recommended benign payloads for classroom use (examples to prepare):

- **HID: Calculator marker** – types a small, benign command that creates /tmp/omg_demo_marker.txt containing "Lab demo: HID injection". (Instructor-only: payload writes to a local file rather than exfiltrating anything.)
- **Persistence marker** – types a short command that creates a scheduled task / cron entry that echoes "Lab demo" to a file every minute (for demonstration only; instructor must remove after).
- **HTTP upload demo** – typed curl command that posts a small dummy file to the Pi5 HTTP sink (http://10.50.0.10:8080/upload) – Pi5 receive directory is /opt/usb_lab/exfil/. Use small dummy data and ensure the Pi5 sink is offline-only.
- **Wireless trigger (meta-demo)** – configure the device to accept remote trigger from a controller on VLAN 50; when triggered it runs a benign action. For classroom safety, simulate wireless triggers if radios are restricted.

DO NOT include payloads that harvest credentials, open shells, move laterally, or exfiltrate real data. Keep everything demonstrably harmless and reversible.

5.2.5 Verification & evidence collection (what students will do)

- Instructor demonstrates payload trigger while students collect artifacts: lsusb, dmesg/journalctl, ps/tasklist, netstat/ip a on the victim; Pi5 logs /opt/usb_lab/logs/* and PCAPs.
- Students must archive: baseline vs after lsusb, dmesg excerpts, the Pi5 exfil folder listing (if payload 3 used), and screenshots of the device Web UI showing payload names/version.
- Forward all logs to IRM: `rsync /opt/usb_lab/logs/ instructor@10.50.0.20:/var/forensics/lab0/<TS>/.`

Quick instructor checklist (copy into the manual)

- Confirm procurement and instructor authorization for device.
- Prepare bench: isolated VLAN, instructor laptop, programmer, Pi5 staging host.
- Vet payloads; compute and record checksums.
- Flash device via Web Flasher (instructor only). Record firmware and SSID/password.
- Upload vetted payloads, verify in device UI. Screenshot About / Payload listing.
- Configure device Wi-Fi (lab-only SSID) or disable AP; store programmer/laptop securely.
- Run demonstration; collect artifacts (Pi5 logs, victim logs, PCAP); forward to IRM.

Appendix note (what to include in Appendix A/B)

- Link to the Web Flasher and firmware docs (the four links you gave).
- usb_watch.sh and USBWatch.ps1 scripts (defensive triage) – mention them as tools students will use to detect the demo.
- A short sample of safe payload files (marker text, small curl uploader) with checksums. **Do not** include complex or dual-use payloads in the student appendix.

 Payload 1: HID keystroke injection (e.g., launch calculator)

Check section 5.2.4

 Payload 2: Automated file access / dummy exfiltration

Check section 5.2.4

 Payload 3: Wireless control of O.MG cable

Check section 5.2.4

6 Detection & Defense (Blue-Team Lens)

In Lab 0 you saw the attacker's toy (the O.MG cable) in action. This section teaches the **defender's** side: how to spot the cable's behaviour, collect reliable evidence, raise alerts, and respond. We focus on host, USB-bus, and network indicators, practical commands you can run on the Pi 5 and on a Windows/Linux victim, and example alert logic you could implement in a SIEM.

6.1 What to look for – high-level indicators

When an O.MG cable is used it usually produces one or more of these observable artefacts:

- **USB enumeration events:** new VID:PID and device class (HID, Mass Storage, RNDIS/CDC).
- **Rapid synthetic input:** processes or applications started at the same moment a keyboard device appears.
- **New block devices / autoruns:** a /dev/sdX appears or a new drive letter is mounted.
- **New network interfaces:** usb0, ethX that correspond to RNDIS/CDC; DHCP or ARP traffic from an unexpected MAC.
- **Unexpected outgoing connections:** short HTTP/TCP POST flows to an internal staging host (Pi5) or external IPs.
- **Wireless beacons:** transient SSIDs or probe activity if the cable exposes an AP.
- **Keystroke/log traces:** typed commands that spawn shells or create files in a short burst.
- **Process creation + device event correlation:** process created within seconds of a USB attach event.

6.2 Host commands & quick triage (what you run in the lab)

On the Raspberry Pi 5 (victim or staging host – Linux)

Capture baseline and then compare after plug-in:

```
# Baseline (before)
lsusb > /tmp/baseline_lsusb.txt
dmesg | tail -n 200 > /tmp/baseline_dmesg.txt
ip -br addr > /tmp/baseline_ip.txt
lsblk -a > /tmp/baseline_lsblk.txt
sudo journalctl -k -n 200 > /tmp/baseline_kernel_journal.txt

# After attach (immediately after)
lsusb > /tmp/after_lsusb.txt
dmesg | tail -n 200 > /tmp/after_dmesg.txt
ip -br addr > /tmp/after_ip.txt
lsblk -a > /tmp/after_lsblk.txt
sudo journalctl -k -n 200 > /tmp/after_kernel_journal.txt

# Quick diffs
diff /tmp/baseline_lsusb.txt /tmp/after_lsusb.txt
diff /tmp/baseline_dmesg.txt /tmp/after_dmesg.txt
```

Extra tools:

- usb-devices (provides descriptors), udevadm monitor --udev --subsystem-match=usb, sudo modprobe usbmon and capture usbmon → open in Wireshark for raw USB frames.

On a Linux victim VM

Same as Pi commands above; also check:

```
journalctl -S "5 minutes ago" | grep -i usb
sudo cat /var/log/auth.log | tail -n 200 # if a programmatic shell was opened
```

On a Windows victim (administrator)

Run PowerShell (elevated):

```
# Baseline
Get-PnpDevice | Out-File C:\lab\baseline_pnp.txt
Get-WinEvent -LogName System -MaxEvents 200 | Out-File C:\lab\baseline_system.txt
Get-Process | Out-File C:\lab\baseline_procs.txt
# After attach
Get-PnpDevice | Out-File C:\lab\after_pnp.txt
Get-WinEvent -LogName System -MaxEvents 200 | Out-File C:\lab\after_system.txt
# Sysmon (if installed)
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 200 | Out-File C:\lab\sysmon_after.txt
```

Look in **Event Viewer** for PnP device events, DriverInstall messages, and ProcessCreate (Sysmon) events.

6.3 Network / Wi-Fi detection (what to capture & filters)

Capture points: IRM sensor, Pi5 (if it's the staging sink), and any wireless monitor in the room.

Quick tcpdump examples (Pi5 / IRM collector):

```
# Capture any RNDIS/DHCP traffic for a short window
sudo timeout 60 tcpdump -i any -w /tmp/labnet_capture.pcap 'port 67 or port 68 or arp or icmp or tcp'
# Capture HTTP POSTs to Pi5 sink
sudo timeout 120 tcpdump -i any -w /tmp/pi5_exfil.pcap 'host 10.50.0.10 and tcp port 8080'
```

Wireshark display filters to use:

- DHCP / BOOTP: bootp
- HTTP POSTs: http.request.method == "POST"
- ARP traffic: arp
- 802.11 probe requests: wlan.fc.type_subtype == 0x04

Wireless: monitor for short-lived SSIDs named O.MG or for probe requests from unknown MACs; correlate timestamps to host events.

6.4 Example SIEM / EDR detection logic (starter rules)

Rule A: Unknown USB device + process creation (pseudo-Splunk)

```
index=host_logs sourcetype=kernel OR sourcetype=Sysmon
| transaction host maxspan=10s startswith="USB device attached" endsuffix="ProcessCreate"
| where searchmatch("idVendor") OR searchmatch("New USB device")
| stats values(ProcessName) as procs, values(USB_Info) by host, transaction_id
| where match(procs, "calc.exe|bash|sh|powershell")
```

6.5 Immediate incident playbook (student / IRM-style)

If you detect a suspicious USB event consistent with an O.MG demo:

1. Contain (fast)

- o Physically unplug the device (instructor does this) OR isolate host (take network down for that VM).
- o If host is critical, isolate via switchport or cloud console.

2. Preserve evidence (do not power off without imaging plan)

- o Save RAM if needed; otherwise take snapshots and collect live logs first. Copy /var/log/, dmesg, lsusb, usbmon pcap, and ps/Event logs. Use scp/rsync to IRM.

3. Collect

- o On Pi5: /opt/usb_lab/logs/, usb_watch outputs, PCAPs.
- o On victim: dmesg, lsusb, journalctl, Windows Event logs, Sysmon logs, any files created by payload.
- o

4. Triage & analyze

- o Correlate timestamps across sources (USB attach → process creation → network flows).
- o Identify VID:PID and lookup vendor OUI / product strings to confirm device identity.
- o Check for persistence artifacts (cron, scheduled tasks, startup folder, registry run keys).

5. Eradicate & recover

- o Remove persistence entries, rebuild host from a known good image, change credentials if risk of compromise.
- o Replace any compromised hardware from secure stock if device cannot be trusted.

6. Report & lessons learned

- o Document timeline, evidence, and mitigation steps in the incident ticket. Add IoCs (VID:PID, MAC addresses, sample payload hashes) to the IRM archive and to the course case study.

6.6 Student lab exercises (blue-team tasks)

These are short hands-on tasks to complete after a demo run. Each task maps to grading points.

Exercise A – Detect the HID injection

- Inputs: instructor runs the HID demo.
- Tasks:
 - Collect `lsusb` before/after, `dmesg` excerpt, and process list showing `calc.exe` or `gnome-calculator`.
 - Produce a 1-page timeline correlating USB attach with process creation.
- Grading points: correct capture & correlation (30%), identification of VID:PID (25%), suggested mitigation (20%).
-

Exercise B – Spot the dummy exfil

- Inputs: instructor triggers the HTTP upload demo to Pi5 sink.
- Tasks:
 - Download Pi5 `/opt/usb_lab/exfil/` listing and the IRM PCAP. Use Wireshark to show the POST.
 - Produce the HTTP request/response snippet and compute SHA256 of the received file.
- Grading: correct PCAP filter and file hash (40%), timeline (30%), recommended network control (20%).
-

Exercise C – Wireless trigger analysis

- Inputs: instructor triggers wireless demo or simulated trigger.
- Tasks:
 - Provide the Wireshark 802.11 capture filtered to probe/beacon frames and show timestamps matching host logs.
 - Suggest two wireless-focused controls for IRM.
- Grading: capture correctness (40%), correlation (30%), mitigation suggestions (20%).

7 Conclusion & Wrap-Up

In this first workshop, you have seen how everyday hardware can be transformed into an attack vector. The **Raspberry Pi 5** acted as our staging and monitoring host, collecting logs and providing a safe observation point. The **O.MG cable**, which at first glance looks like an ordinary charging cord, revealed its hidden capabilities once connected to the victim system.

Behind the scenes, your instructors carefully prepared this environment: the O.MG cable was safely flashed using the official programmer, its firmware verified, and only **benign payloads** were uploaded in advance. These payloads – such as a harmless HID command that launches Calculator, a persistence marker that leaves a visible trace, and a dummy HTTP upload to the Pi5 – were chosen to give you observable artifacts without causing harm. That preparation ensured you could focus today on recognizing suspicious USB behavior, collecting forensic evidence, and understanding the flow of an attack.

The larger purpose of **Lab 0** is not to glorify offensive tools, but to demonstrate how they work so that you can better defend against them. This is what ethical penetration testing is about: controlled, authorized exercises that sharpen your awareness of threats and your ability to detect them. By completing this lab, you now understand how malicious hardware can infiltrate a system, what signs it leaves behind, and how defenders can respond. These are the foundations we will build upon in the coming labs as we continue to connect offensive techniques with defensive strategies.

◆ Appendix A

Windows – USBWatch.ps1 (PowerShell)

A PowerShell script that collects baseline Device/Driver lists, monitors the System log for new PnP events, and produces a short comparison. Run in an elevated PowerShell session.

Save as **USBWatch.ps1**, run as Administrator: `.\USBWatch.ps1 -Label studentA_run`.

```
<#
USBWatch.ps1 - baseline + monitor + after snapshot for USB device inspection
Run as Administrator: .\USBWatch.ps1 -Label "test1"
#>

param(
    [string]$Label = "run",
    [int]$MonitorSeconds = 180
)

$TS = (Get-Date).ToUniversalTime().ToString("yyyyMMddTHH:mm:ssZ")
$OutDir = Join-Path -Path $env:TEMP -ChildPath ("USBWatch_{0}_{1}" -f $Label, $TS)
New-Item -ItemType Directory -Path $OutDir | Out-Null
Write-Host "Output: $OutDir"

function Save-Baseline {
    Write-Host "$(Get-Date -Format u) Saving baseline..."
    # PnP devices
    Get-PnpDevice | Sort-Object Class,InstanceId | Out-File -FilePath (Join-Path $OutDir "pnp_baseline.txt")
    # PnP device properties for more detail (may be slow)
    Get-PnpDeviceProperty -KeyName "DEVKEY_Device_HardwareIds" | Out-File (Join-Path $OutDir
"pnp_props_baseline.txt")
    # Network interfaces
    Get-NetAdapter | Format-List * | Out-File (Join-Path $OutDir "net_baseline.txt")
    # Mounted volumes
    Get-Volume | Format-List * | Out-File (Join-Path $OutDir "volumes_baseline.txt")
    # Recent System event log (kernel/pnp)
    Get-WinEvent -LogName System -MaxEvents 200 | Out-File (Join-Path $OutDir "system_baseline_events.txt")
    # Sysmon (if present)
    if (Get-WinEvent -ListLog "Microsoft-Windows-Sysmon/Operational" -ErrorAction SilentlyContinue) {
        Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 200 | Out-File (Join-Path
$OutDir "sysmon_baseline.txt")
    }
}

function Start-Monitoring {
    Write-Host "$(Get-Date -Format u) Starting live monitor for $MonitorSeconds seconds. Plug the cable now..."
    $query = @"
<QueryList>
  <Query Id="0" Path="System">
    <Select Path="System">*[System[(Level=4 or Level=3)]] and *[EventData[Data and (Data='USB' or Data='Device'
or Data='PnP')]]</Select>
  </Query>
</QueryList>
"@

    # Use Get-WinEvent with -FilterXPath or just poll latest System events during the timeframe
    $end = (Get-Date).AddSeconds($MonitorSeconds)
    $sysLogFile = Join-Path $OutDir "system_monitor_events.txt"
    "System monitor started at $(Get-Date -Format u)" | Out-File $sysLogFile
    while ((Get-Date) -lt $end) {
        Get-WinEvent -LogName System -MaxEvents 50 | Select-Object -First 50 | Out-File -Append $sysLogFile
        Start-Sleep -Seconds 2
    }
    Write-Host "$(Get-Date -Format u) Live monitor finished. Saved to $sysLogFile"
```

```

}

function Save-After {
    Write-Host "$(Get-Date -Format u) Saving after-snapshot..."
    Get-PnpDevice | Sort-Object Class,InstanceId | Out-File -FilePath (Join-Path $OutDir "pnp_after.txt")
    Get-PnpDeviceProperty -KeyName "DEVKEY_Device_HardwareIds" | Out-File (Join-Path $OutDir
"pnp_props_after.txt")
    Get-NetAdapter | Format-List * | Out-File (Join-Path $OutDir "net_after.txt")
    Get-Volume | Format-List * | Out-File (Join-Path $OutDir "volumes_after.txt")
    Get-WinEvent -LogName System -MaxEvents 200 | Out-File (Join-Path $OutDir "system_after_events.txt")
    if (Get-WinEvent -ListLog "Microsoft-Windows-Sysmon/Operational" -ErrorAction SilentlyContinue) {
        Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 200 | Out-File (Join-Path
$OutDir "sysmon_after.txt")
    }
}

function Diff-Files {
    Write-Host "$(Get-Date -Format u) Producing diffs..."
    Compare-Object -ReferenceObject (Get-Content (Join-Path $OutDir "pnp_baseline.txt")) `
        -DifferenceObject (Get-Content (Join-Path $OutDir "pnp_after.txt")) |
        Out-File (Join-Path $OutDir "pnp_diff.txt")
    Compare-Object -ReferenceObject (Get-Content (Join-Path $OutDir "net_baseline.txt")) `
        -DifferenceObject (Get-Content (Join-Path $OutDir "net_after.txt")) |
        Out-File (Join-Path $OutDir "net_diff.txt")
    Compare-Object -ReferenceObject (Get-Content (Join-Path $OutDir "volumes_baseline.txt")) `
        -DifferenceObject (Get-Content (Join-Path $OutDir "volumes_after.txt")) |
        Out-File (Join-Path $OutDir "volumes_diff.txt")
    Write-Host "Diffs saved under $OutDir"
}

# Run workflow
Save-Baseline
Start-Monitoring
Save-After
Diff-Files

# Produce quick summary
$summary = Join-Path $OutDir "summary.txt"
@(
    "USBWatch Summary - $TS",
    "Label: $Label",
    "",
    "PNP differences (see pnp_diff.txt):",
    Get-Content (Join-Path $OutDir "pnp_diff.txt") -ErrorAction SilentlyContinue,
    "",
    "Network differences (see net_diff.txt):",
    Get-Content (Join-Path $OutDir "net_diff.txt") -ErrorAction SilentlyContinue,
    "",
    "Volume differences (see volumes_diff.txt):",
    Get-Content (Join-Path $OutDir "volumes_diff.txt") -ErrorAction SilentlyContinue,
    ""
) | Out-File $summary

Write-Host "Done. Output directory: $OutDir"
Write-Host "Review summary: $summary"

```

◆ Appendix B

Linux – usb_watch.sh (bash)

Saves baseline, runs live monitors (udevadm, dmesg, optional usbmon), then produces a short report showing what changed.

Save as **usb_watch.sh**, make executable `chmod +x usb_watch.sh`, then run with `sudo ./usb_watch.sh`.

```
#!/usr/bin/env bash
# usb_watch.sh – baseline + live monitor + after-snapshot + quick analysis
# Usage: sudo ./usb_watch.sh <run_label>
# Example: sudo ./usb_watch.sh studentA_hid_demo

set -euo pipefail

LABEL=${1:-run}
TS=$(date -u +"%Y-%m-%dT%H%M%S")
OUTDIR="/tmp/usb_watch_${LABEL}_${TS}"
mkdir -p "$OUTDIR"
echo "[*] Output directory: $OUTDIR"

# Helper for timestamped log writes
ts() { date -u +"%Y-%m-%dT%H:%M:%S"; }

echo "[`ts`] Collecting baseline info..."
# Baseline: lsusb, lsusb -v (summary), kernel tail, network, mounts, processes
lsusb > "$OUTDIR/lsusb_baseline.txt"
lsusb -v > "$OUTDIR/lsusb_v_baseline.txt" 2>/dev/null || true
dmesg | tail -n 200 > "$OUTDIR/dmesg_baseline.txt"
ip -br addr > "$OUTDIR/ip_baseline.txt"
lsblk -a > "$OUTDIR/lsblk_baseline.txt"
mount | column -t > "$OUTDIR/mount_baseline.txt"
ps aux --sort=start_time | head -n 80 > "$OUTDIR/ps_baseline.txt"

# Optional: whitelist file (VID:PID lines) – populate as needed
WHITELIST="$OUTDIR/vidpid_whitelist.txt"
cat > "$WHITELIST" <<'EOF'
# Example: vendor:product
046d:c31c # Logitech keyboard (example)
05ac:12a8 # Apple device (example)
EOF
echo "[`ts`] Baseline complete."

# Start background monitors: udev events and dmesg tail
echo "[`ts`] Starting live monitors (udev, dmesg). Press ENTER to stop monitoring after plugging device."
( udevadm monitor --udev --subsystem-match=usb 2>&1 | sed "s/^/UDEV: /" ) > "$OUTDIR/udev_monitor.log" 2>&1 &
UDEV_PID=$!
( dmesg -wH 2>&1 | sed "s/^/DMESG: /" ) > "$OUTDIR/dmesg_monitor.log" 2>&1 &
DMESG_PID=$!

# Optional: enable usbmon capture (requires root and may produce large files)
read -p "Start usbmon capture? (y/N): " use_usbmon
if [[ "${use_usbmon,,}" =~ ^y ]]; then
    if ! modprobe usbmon 2>/dev/null; then
        echo "[!] usbmon modprobe failed or not available"
    else
        # choose usbmon0 (sum of all buses). Use usbmonX for specific bus if needed.
        USBMON_IF="usbmon0"
        echo "[`ts`] Starting usbmon capture on $USBMON_IF ..."
        # tshark must be installed. Fall back to tcpdump with -i usbmon0 if not.
        if command -v tshark >/dev/null 2>&1; then
            tshark -i "$USBMON_IF" -w "$OUTDIR/usbmon_capture.pcap" >/dev/null 2>&1 &
            USBMON_PID=$!
        else
            tcpdump -i "$USBMON_IF" -s 0 -w "$OUTDIR/usbmon_capture.pcap" >/dev/null 2>&1 &
            USBMON_PID=$!
        fi
    fi
fi
```

```

fi
fi

echo ""
echo "== INSTRUCTIONS =="
echo "1) Press ENTER to start the interactive phase (then plug the cable into the host).\"
echo "2) When observations are complete, come back and press ENTER again to finish.\"
read -p \"Ready? Press ENTER to start interactive monitoring...\"

echo "[`ts`] Interactive monitoring started. Please plug the cable now and observe. Hit ENTER when done.\"
read -p \"Press ENTER after you have finished the interaction with the device...\"

# Stop monitors
echo "[`ts`] Stopping monitors...\"
kill $UDEVPID $DMESGPID 2>/dev/null || true
wait $UDEVPID $DMESGPID 2>/dev/null || true
if [[ -n \"${USBMON_PID:-}\" ]]; then
    kill $USBMON_PID 2>/dev/null || true
    wait $USBMON_PID 2>/dev/null || true
fi

# Collect after-snapshot
echo "[`ts`] Collecting after-snapshot...\"
lsusb > \"$OUTDIR/lsusb_after.txt\"
lsusb -v > \"$OUTDIR/lsusb_v_after.txt\" 2>/dev/null || true
dmesg | tail -n 500 > \"$OUTDIR/dmesg_after.txt\"
ip -br addr > \"$OUTDIR/ip_after.txt\"
lsblk -a > \"$OUTDIR/lsblk_after.txt\"
mount | column -t > \"$OUTDIR/mount_after.txt\"
ps aux --sort=start_time | head -n 120 > \"$OUTDIR/ps_after.txt\"

# Quick diff summaries
echo "[`ts`] Producing quick diffs...\"
comm -3 <(sort \"$OUTDIR/lsusb_baseline.txt\") <(sort \"$OUTDIR/lsusb_after.txt\") > \"$OUTDIR/lsusb_diff.txt\" ||
true
echo "[`ts`] lsusb differences saved to $OUTDIR/lsusb_diff.txt\"

# Detect HID / storage / RNDIS by scanning for common keywords in verbose output
grep -i -E \"Human Interface|HID|Keyboard|Mouse|Mass Storage|Mass-storage|MassStorage|CDC|RNDIS|ECM|CDC-
ECM|usb0|rndis|vendor|idProduct|idVendor\" \"$OUTDIR/lsusb_v_after.txt\" > \"$OUTDIR/lsusb_v_flags.txt\" || true

# Detect new block devices
comm -3 <(sort \"$OUTDIR/lsblk_baseline.txt\") <(sort \"$OUTDIR/lsblk_after.txt\") > \"$OUTDIR/lsblk_diff.txt\" ||
true

# Detect new ip addresses / interfaces
comm -3 <(sort \"$OUTDIR/ip_baseline.txt\") <(sort \"$OUTDIR/ip_after.txt\") > \"$OUTDIR/ip_diff.txt\" || true

# Flag unknown VID:PID (very simple)
awk '{print $6,$7,$0}' \"$OUTDIR/lsusb_after.txt\" | awk -FID '{print $2}' | awk '{print $1}' >
\"$OUTDIR/vidpid_after_only.txt\" || true
# Compare to whitelist
echo \"Unknown VID:PID (not in whitelist):\" > \"$OUTDIR/unknown_vidpid.txt\"
while read -r vidpid; do
    if ! grep -q -i \"$vidpid\" \"$WHITELIST\"; then
        echo \"$vidpid\" >> \"$OUTDIR/unknown_vidpid.txt\"
    fi
done < \"$OUTDIR/vidpid_after_only.txt\" || true

# Summary report
REPORT=\"$OUTDIR/summary_report.txt\"
{
    echo \"USB Watch Summary - $TS\"
    echo \"Run label: $LABEL\"
    echo \"\"
    echo \"lsusb diff (new devices):\"
    sed -n '1,200p' \"$OUTDIR/lsusb_diff.txt\" || true
    echo \"\"
    echo \"Potential flags from verbose lsusb (lsusb -v):\"
    sed -n '1,200p' \"$OUTDIR/lsusb_v_flags.txt\" || true
}

```

```
echo ""
echo "New block devices (lsblk diff):"
sed -n '1,200p' "$OUTDIR/lsblk_diff.txt" || true
echo ""
echo "New network interfaces / IPs detected:"
sed -n '1,200p' "$OUTDIR/ip_diff.txt" || true
echo ""
echo "Unknown VID:PID (compared to simple whitelist $WHITELIST):"
sed -n '1,200p' "$OUTDIR/unknown_vidpid.txt" || true
echo ""
echo "Note: inspect $OUTDIR/dmesg_after.txt, $OUTDIR/udev_monitor.log, and $OUTDIR/dmesg_monitor.log for
kernel messages and timing correlation."
} > "$REPORT"

echo "[`ts`] Report written to $REPORT"
echo "[*] Done. Tarballing outputs..."
tar czf "${OUTDIR}.tgz" -C /tmp "${basename "$OUTDIR"}"
echo "[*] Archive created: ${OUTDIR}.tgz"
echo "[*] Inspect logs and escalate if unknown or suspicious devices detected."
```